PROJECT AND TEAM INFORMATION

Project Title

Emergency Call Detection and Priority Routing

Team Name: *Team #*	*NOVEXA*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Dixit, Avika :2510018349* *avika51110@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Bisht, Harshita :2510360038* *bishtharshi25@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Mittal, Saksham : 2510014103* *sakshammittal.geu@gmail.c* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Bhandari, Nitin Singh :2512510026* *nitinsinghbhandari@gmail.c* 

m

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Our project is about creating an Emergency Call Detection and Priority Routing System.. The main goal of the project is to make the way emergency reports are handled organized and help emergency teams act more quickly. In an emergency, many people might call and report the same situation .Some of these reports might have the details; some might be missing important information and some might even have different details. If each call is treated as a case it can cause confusion and make the job harder for the people handling the calls.

Our system processes incoming emergency reports and checks whether they are related to an incident that has already been reported. If several reports are about the event they can be grouped together instead of being treated as separate cases. The system can also give attention to reports that don't have much useful information. Another important part of the project is assigning priorities to incidents based on their urgency and severity. Incidents are ranked based on how serious they're. Dangerous situations are placed higher in the list so that emergency teams can deal with them first.

The system uses different data structures to manage emergency reports. A queue processes incoming reports in the order they are received.A hash map is used to store and find emergency reports. Graphs and DSU can be used to find links between reports and put them into groups. A priority queue is used to organise incidents according to their urgency. The project is developed in C++ and uses object-oriented programming concepts such as classes, inheritance, and polymorphism, along with the Standard Template Library (STL) .In terms the project takes many emergency reports groups the ones that are related into incident gives higher priority to more serious situations and helps the operator decide which emergency to handle first..
In Phase III, the project has been completed with dynamic input handling and an improved user interface. Users can submit emergency reports by entering the required details and uploading photographs as supporting evidence.

Updated Project Approach and Architecture

The project "Emergency Call Detection & Priority Routing" is currently developed as console based .The main objective of the project focuses on managing incoming emergency reports by identifying duplicate or related calls, combining them into a single incident, and assigning priorities based on urgency. This helps responders focus on critical situations and manage emergencies more efficiently.

The current architecture follows object-oriented approach. Different classes are used for major entities such as EmergencyCall, Incident, Caller, Dispatcher, ResponseTeam, and CallAnalyzer. Data structures are integrated into the system according to their requirements. A Hash Map is used to identify duplicate calls or reports, Graph/Disjoint Set concepts are used to group related calls into incidents, a Priority Queue is used to prioritize critical incidents, and a Queue represents incoming emergency calls.

The CallAnalyzer is the part that handles communication with other classes. It analyzes calls. Updates incidents accordingly. The Dispatcher takes the processed data. Determines the priority level of each incident based on its urgency and severity.Then it assigns the response teams based on the urgency.

In Phase III, the project was completed with dynamic input handling and an improved user interface. Users can now submit emergency reports by entering details such as location, emergency type, description, and severity. The system also allows users to upload photographs as evidence along with their reports. Once submitted, the reports are stored in the system, and users can view their previous submissions through the report history section.

Tasks Completed

Task Completed	Team Member
Project Design and System Workflow	Saksham Mittal
Data Structure and Incident Grouping Implementation	Avika Dixit
Emergency Report Handling and Call Analysis Implementation	Harshita Bisht
Incident Prioritization and Response Team Implementation	Nitin Singh Bhandari

Challenges/Roadblocks

During Phase III, our team faced some problems while completing the project and connecting all the modules together. Handling of dynamic inputs and linking them with the existing classes was a bit little difficult task and it took more time than that what we expected. We had also faced some difficulty in combining different data structures like Queue, Hash Map, Graph, DSU and Priority Queue into a single working system a bit challenging.
While developing the project, we also encountered some compilation errors like header file problems and issues in connecting different C++ classes. Apart from the main implementation, working on the dashboard and making the process of report submission function properly and efficiently also required extra efforts. Some features like uploading photos of the incident, showing the submitted reports and keeping a record of all the reports also took a lot of time to complete and work properly.
We solved all these problems by re-checking the code, debugging all the encountered errors and testing each module several times. Some of the changes were made whenever something was not working as we expected. With teamwork, proper coordination and continuous efforts of all the team members, we were finally able to solve the most of the issues and complete the project successfully within the time limit.

Tasks Pending

Task Pending	Team Member
Making the system dynamic instead of static.	Saksham Mittal
Connecting the system features so that reports and incidents can be updated.	Nitin Singh Bhandari
Working on the UI/UX and designing the dashboard.	Avika Dixit
Improving the dashboard layout, colours, and overall look.	Harshita Bisht

Project Outcome/Deliverables

The main outcome of our project is an Emergency Call Detection and Priority Routing System that helps emergency services manage incoming reports more efficiently.

The key deliverables are:

1. Emergency Report Management: A FIFO(first in first out) queue-based system to receive and process emergency reports efficently.
2. Low-Information Call Detection: A call analyzer class is defined to identify incomplete or low information reports.
3. Incident Grouping: The use of hash maps, graphs and disjoint set union(DSU) to identify and combine the related reports into a single incident.
4. Incident Prioritization: A priority queue decides which emergency incident should be handled first.
5. Response Management: A system workflow that helps determine which incidents should be handled first and which support team should dispatch the teams.
6. C++ Implementation: Integration of the system ,into the existing infrastructure.

Progress Overview

The project has been successfully completed in Phase III. During Phase I, the team worked on identifying the problem, understanding the project requirements, and planning of the basic workflow of the system.

In Phase II, the main C++ code was developed, along with the required classes and data structures. A basic console-based prototype with constant inputs was also created and tested using predefined inputs.

In Phase III, our team completed the remaining work of the project. We added dynamic input handling and connected all the major modules together. The user interface was also improved so that users can easily understand and use the system.
The final system allows users to submit emergency reports and process the given details. It can identify duplicate or related reports and group them into incidents. The system also gives priority according to the urgency and seriousness of the situation and selects a suitable response team.
The dashboard provides options to submit emergency reports, upload photographs as proof and view previously submitted reports. This makes the whole reporting process more simple and organized for users.
Overall, while working on this project, our team learned how Data Structures and Object-Oriented Programming can be used to solve real-life problems. We also got practical experience in connecting different modules and handling errors. After completing the planned features and making the required changes, we successfully completed our final working prototype.

CODEBASE INFORMATION

Programming Language: C++

IDE: Visual Studio Code

Version Control: Git and GitHub

Repository: https://github.com/avikad05/PBL-3RD-SEM

COMMITS

Caller.h – Stores the details of the caller, such as caller ID, name and phone number.

EmergencyCall.h – It contains the details of each emergency call, including the location and type of emergency, severity and the timing of the call.

Incident.h –It represents a combined incident that consist of all the related emergency individual calls along with their priority.

DSU.h – It uses the Disjoint Set Union (DSU) data structure to group related emergency calls under the same incident group.

CallAnalyzer.h – It analyses and checks all the incoming calls and finds the related reports, identifies calls with less information and calculates the priority of each incident.

ResponseTeam.h – It defines different type of emergency teams, such as Police , Ambulance and fire teams.

Dispatcher.h – It selects and also assigns the most suitable response team according to the type of emergency and the priority of the emergency.

Main.cpp – It Controls the final execution of the program and connects all the different classes and modules to make the system work.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Queue Testing	**Pass**	Incoming emergency calls is processed in first in first out queue order.
Duplicate call detection	**Pass**	Hash map identifies calls with matching emergency types and locations
Incident grouping	**Pass**	DSU groups related emergency calls based on defines conditions
Low information detection	**Pass**	Calls with missing insufficient information are flagged
Priority queue testing	**Pass**	Higher priority incidents are given precedence processing
Response team dispatching	**Pass**	Appropriate teams are selected according to the emergency type.

Deliverables Progress

DELIVERABLE	STATUS
Handling of Emergency Reports using Queue	COMPLETED
Analysis of Reports and Identification of Related Reports	COMPLETED
Incident Grouping using Hash Map, Graph and DSU	COMPLETED
Incident Prioritization using Priority Queue	COMPLETED
Team and Dispatch Process	COMPLETED
Working Prototype – Static Input	COMPLETED
Dynamic Input Handling	COMPLETED
Final Testing and Integration	COMPLETED
UI/UX dashboard	COMPLETED